

Extra Questions: Distance Matrix Computation

Demonstration of Distance Matrix Computation using Multiple Distance Metrics on Student Survey Dataset

Student ID: 2547125

Course: Machine Learning Lab

Dataset: Student Survey Dataset (data.xlsx)

Aim

To demonstrate distance matrix computation using two distance calculation methods (Euclidean and Manhattan) on the Student Survey dataset. The workflow includes data cleaning and preprocessing, exploratory data analysis, feature standardization, custom distance calculations (vectorized and iterative), validation using SciPy, and distance visualization through heatmaps.

Dataset Overview

The dataset contains survey responses from 163 students with 14 variables representing academic performance (GPA, CIA %), attendance, technical competencies, extracurricular activities, and placement metrics/expectations.

Problem Statement

Distance-based algorithms (such as KNN, K-Means, and Hierarchical Clustering) rely heavily on distance metrics to identify similarities. We are required to clean the raw student survey data, standardize the numerical features to ensure equal contribution, compute the pairwise distance matrices using Euclidean and Manhattan distance metrics, visualize the resulting similarity profiles, and analyze the mathematical and empirical differences between the two methods.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import re
from datetime import datetime
from scipy.spatial.distance import cdist
```

```

from sklearn.preprocessing import StandardScaler

# Set plot styles
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (9, 5)
plt.rcParams["font.size"] = 10
print("Libraries imported successfully!")

```

Libraries imported successfully!

Part A: Data Preprocessing & Cleaning (Task 1)

1. Load Data and Check Structure

I will load the dataset from the Excel file `data.xlsx` and inspect its shape and raw structure.

```

In [2]: # Load dataset
df = pd.read_excel('data.xlsx')

print(f"Dataset shape: {df.shape[0]} rows, {df.shape[1]} columns")
print("\nRaw columns:")
for i, col in enumerate(df.columns):
    print(f" {i}: {col}")

```

Dataset shape: 163 rows, 14 columns

Raw columns:

```

0: Timestamp
1: Reg No
2: Job role that you are interested in
3: What is the minimum salary of students placed through campus (In LPA..respond a
s a number)
4: What is the maximum salary of students placed through campus (In LPA..respond a
s a number)
5: What is the median salary of students placed through campus (In LPA..respond a
s a number)
6: Which is the highest paying company
7: Rate your contribution towards extra curricular activities
8: Rate your technical competencies
9: What are your package expectations (LPA)
10: your CIA % of last semester
11: your GPA of last semester
12: Your maximum attendance % till last semester
13: Internships Interests

```

2. Rename Columns

For convenience and readability, I will rename the columns to shorter, more manageable names.

```

In [3]: # Rename columns
df.columns = [

```

```

    'Timestamp', 'Reg_No', 'Job_Role', 'Min_Salary_Placed', 'Max_Salary_Placed',
    'Median_Salary_Placed', 'Highest_Paying_Company', 'Extra_Curricular', 'Technica
    'Package_Expectations', 'CIA_Percent', 'GPA', 'Attendance_Percent', 'Internship
]

print("First 5 rows of renamed dataset:")
display(df.head())

```

First 5 rows of renamed dataset:

	Timestamp	Reg_No	Job_Role	Min_Salary_Placed	Max_Salary_Placed	Median_Salary_Plac
0	2024-12-04 20:31:09	2341338	Data Scientist	3 LPA	1Cr	20 L
1	2024-12-04 20:33:07	2341310	Data Scientist	5	NaN	N
2	2024-12-04 20:39:43	2341311	Data Scientist	7 LPA	15 LPA	8 L
3	2024-12-04 20:40:37	2341324	Data Scientist	3 LPA	25 LPA	10 L
4	2024-12-04 20:41:51	2341324	Business Analyst	2	50	

3. Clean and Standardize Data Types

The dataset contains several text entries, formatting variations, and Excel datetime parsing errors. I will implement custom cleaning functions for the salary, GPA, CIA %, and attendance columns.

```

In [4]: def clean_salary(val):
        if pd.isna(val):
            return np.nan
        if isinstance(val, (datetime, pd.Timestamp)):
            # Excel converts ranges like '5-6' to datetime objects.
            # We average the month and day to recover the approximate midpoint.
            return (val.month + val.day) / 2.0

        s = str(val).strip().lower()
        # Handle descriptive non-numeric entries
        if any(x in s for x in ['high', 'nothing', 'expect', 'morbillion', 'job securit
            return np.nan

        # Handle ranges represented as string (e.g. '10 - 14 lpa')
        if '-' in s:
            parts = re.findall(r'[\d\.]+', s)
            if len(parts) >= 2:
                try:
                    return (float(parts[0]) + float(parts[1])) / 2.0
                except:
                    pass

```

```

        return np.nan

    # Check for Crores (1 Cr = 100 LPA)
    is_crore = 'cr' in s
    # Remove commas and non-numeric Letters (Like LPA, Lac, etc.)
    s = s.replace(',', '')
    s = re.sub(r'^\d\.', '', s)
    if not s:
        return np.nan
    try:
        x = float(s)
        if is_crore:
            return x * 100.0

        # Scale handling:
        # If absolute Rupee amount >= 10,000,000 (1 Cr), divide by 1,000,000 to get
        # If absolute Rupee amount >= 100,000, divide by 100,000 to get LPA
        # If absolute Rupee amount >= 10,000 (monthly salary or typo), divide by 10
        # If absolute Rupee amount >= 1,000, divide by 1,000 to get LPA
        if x >= 10000000:
            return x / 1000000.0
        elif x >= 100000:
            return x / 100000.0
        elif x >= 10000:
            return x / 10000.0
        elif x >= 1000:
            return x / 1000.0
        return x
    except:
        return np.nan

def clean_cia(val):
    if pd.isna(val):
        return np.nan
    s = str(val).strip().lower()
    if s in ['ongoing', '-', '']:
        return np.nan
    s = s.replace('%', '')
    try:
        x = float(s)
        # Handle fractional format (e.g., 0.8 -> 80.0)
        if x <= 1.0:
            return x * 100.0
        # Handle single digit values representing a 10-point scale (e.g., 8 -> 80%)
        elif x <= 10.0:
            return x * 10.0
        return x
    except:
        return np.nan

def clean_gpa(val):
    if pd.isna(val):
        return np.nan
    if isinstance(val, (datetime, pd.Timestamp)):
        return (val.month + val.day) / 2.0
    s = str(val).strip().lower()

```

```

if s in ['ongoing', '-', '']:
    return np.nan
if '/' in s:
    parts = s.split('/')
    try:
        return (float(parts[0]) / float(parts[1])) * 10.0
    except:
        return np.nan
try:
    x = float(s)
    if x <= 1.0:
        # Fraction (e.g. 0.8 -> 8.0 out of 10)
        return x * 10.0
    elif x <= 4.0:
        # 4-point GPA scale to 10-point scale
        return x * 2.5
    elif x <= 10.0:
        return x
    else:
        # Percentage (e.g. 80 -> 8.0 out of 10)
        return x / 10.0
except:
    return np.nan

def clean_attendance(val):
    if pd.isna(val):
        return np.nan
    s = str(val).strip().lower()
    if '100' in s:
        return 100.0
    try:
        x = float(s)
        if x <= 1.0:
            return x * 100.0
        return x
    except:
        return np.nan

# Copy DataFrame for cleaning
df_clean = df.copy()

# Apply cleaning functions
df_clean['Min_Salary_Placed'] = df_clean['Min_Salary_Placed'].apply(clean_salary)
df_clean['Max_Salary_Placed'] = df_clean['Max_Salary_Placed'].apply(clean_salary)
df_clean['Median_Salary_Placed'] = df_clean['Median_Salary_Placed'].apply(clean_sal
df_clean['Package_Expectations'] = df_clean['Package_Expectations'].apply(clean_sal
df_clean['CIA_Percent'] = df_clean['CIA_Percent'].apply(clean_cia)
df_clean['GPA'] = df_clean['GPA'].apply(clean_gpa)
df_clean['Attendance_Percent'] = df_clean['Attendance_Percent'].apply(clean_attenda

# Impute obvious typos
# Row 139 median salary was 200, which is an obvious typo for 20
df_clean.loc[df_clean['Median_Salary_Placed'] == 200, 'Median_Salary_Placed'] = 20.

print("Cleaned summary stats (pre-imputation):")
display(df_clean[['Min_Salary_Placed', 'Max_Salary_Placed', 'Median_Salary_Placed',

```

Cleaned summary stats (pre-imputation):

	Min_Salary_Placed	Max_Salary_Placed	Median_Salary_Placed	Package_Expectations	C
count	162.000000	158.000000	157.000000	157.000000	
mean	6.296296	24.428987	11.603471	18.141401	
std	6.230698	18.538060	9.480128	63.934251	
min	1.000000	1.000000	1.200000	3.000000	
25%	3.000000	12.000000	6.000000	7.000000	
50%	5.000000	20.000000	9.000000	10.000000	
75%	7.000000	30.000000	14.000000	15.000000	
max	66.000000	100.000000	76.000000	800.000000	

4. Missing Value Imputation

I will check for any remaining null values in the cleaned numerical columns and impute them using their respective medians.

```
In [5]: features = [  
    'Min_Salary_Placed', 'Max_Salary_Placed', 'Median_Salary_Placed',  
    'Package_Expectations', 'CIA_Percent', 'GPA', 'Attendance_Percent',  
    'Extra_Curricular', 'Technical_Competency'  
]  
  
print("Missing values count per column before imputation:")  
print(df_clean[features].isna().sum())  
  
# Impute missing values with column medians  
imputation_map = {}  
for col in features:  
    median_val = df_clean[col].median()  
    df_clean[col] = df_clean[col].fillna(median_val)  
    imputation_map[col] = median_val  
  
print("\nMissing values count per column after imputation:")  
print(df_clean[features].isna().sum())
```

Missing values count per column before imputation:

Min_Salary_Placed	1
Max_Salary_Placed	5
Median_Salary_Placed	6
Package_Expectations	6
CIA_Percent	13
GPA	9
Attendance_Percent	5
Extra_Curricular	1
Technical_Competency	0

dtype: int64

Missing values count per column after imputation:

Min_Salary_Placed	0
Max_Salary_Placed	0
Median_Salary_Placed	0
Package_Expectations	0
CIA_Percent	0
GPA	0
Attendance_Percent	0
Extra_Curricular	0
Technical_Competency	0

dtype: int64

My Observation:

- **Data Entry Errors & Cleanup:** The raw dataset exhibits significant formatting variations:
 - Textual suffixes like " LPA", " Lakhs", or " lac" were stripped and standardized to LPA.
 - Excel parsed range entries (such as "5-6") as dates (e.g. May 6, 2026). These were successfully recovered by computing the midpoint of the month and day.
 - A student reported a median placement salary of 200 LPA which is an outlier for the companies placed (EY, etc.); it was corrected to 20.0 LPA.
 - CIA % and GPA values included scale inconsistencies: CIA % was represented both as a percentage (e.g. 80) and as a fraction (e.g. 0.8), while GPA was entered as percentage (80), out of 4 (3.5), or out of 10 (8.5). All GPAs were standardized to a 10-point scale and CIA % to a 100-point scale.
 - Attendance % fractions (e.g. 0.9) were converted to full percentages (90%).
- **Imputation Strategy:** Standardizing the numerical fields before imputing allowed us to compute clean medians without scale distortion. Features such as CIA_Percent (13 missing) and GPA (9 missing) were successfully filled.

Part B: Exploratory Data Analysis (EDA) (Task 2)

I will plot the distributions of key variables to visualize the academic profiles and career expectations of the students in the dataset.

```
In [6]: # Create plots folder if it doesn't exist
os.makedirs('plots', exist_ok=True)
```

```

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. GPA distribution
sns.histplot(df_clean['GPA'], kde=True, ax=axes[0, 0], color='royalblue')
axes[0, 0].set_title('Distribution of GPA (Standardized to 10-Point Scale)')
axes[0, 0].set_xlabel('GPA')

# 2. Attendance distribution
sns.histplot(df_clean['Attendance_Percent'], kde=True, ax=axes[0, 1], color='teal')
axes[0, 1].set_title('Distribution of Attendance %')
axes[0, 1].set_xlabel('Attendance %')

# 3. Expected Package vs. Median Salary
sns.kdeplot(df_clean['Package_Expectations'], label='Expected Salary (LPA)', fill=True)
sns.kdeplot(df_clean['Median_Salary_Placed'], label='Median Placed Salary (LPA)', fill=True)
axes[1, 0].set_title('Comparison: Salary Expectations vs. Campus Placed Median')
axes[1, 0].set_xlabel('LPA')
axes[1, 0].legend()

# 4. Job Role Interests Count
sns.countplot(y=df_clean['Job_Role'].fillna('Not Specified'), order=df_clean['Job_Role'].value_counts().index, ax=axes[1, 1])
axes[1, 1].set_title('Student Interest by Job Role')
axes[1, 1].set_xlabel('Count')
axes[1, 1].set_ylabel('Job Role')

plt.tight_layout()
plt.savefig('plots/eda_distributions.png', dpi=300)
plt.show()

```

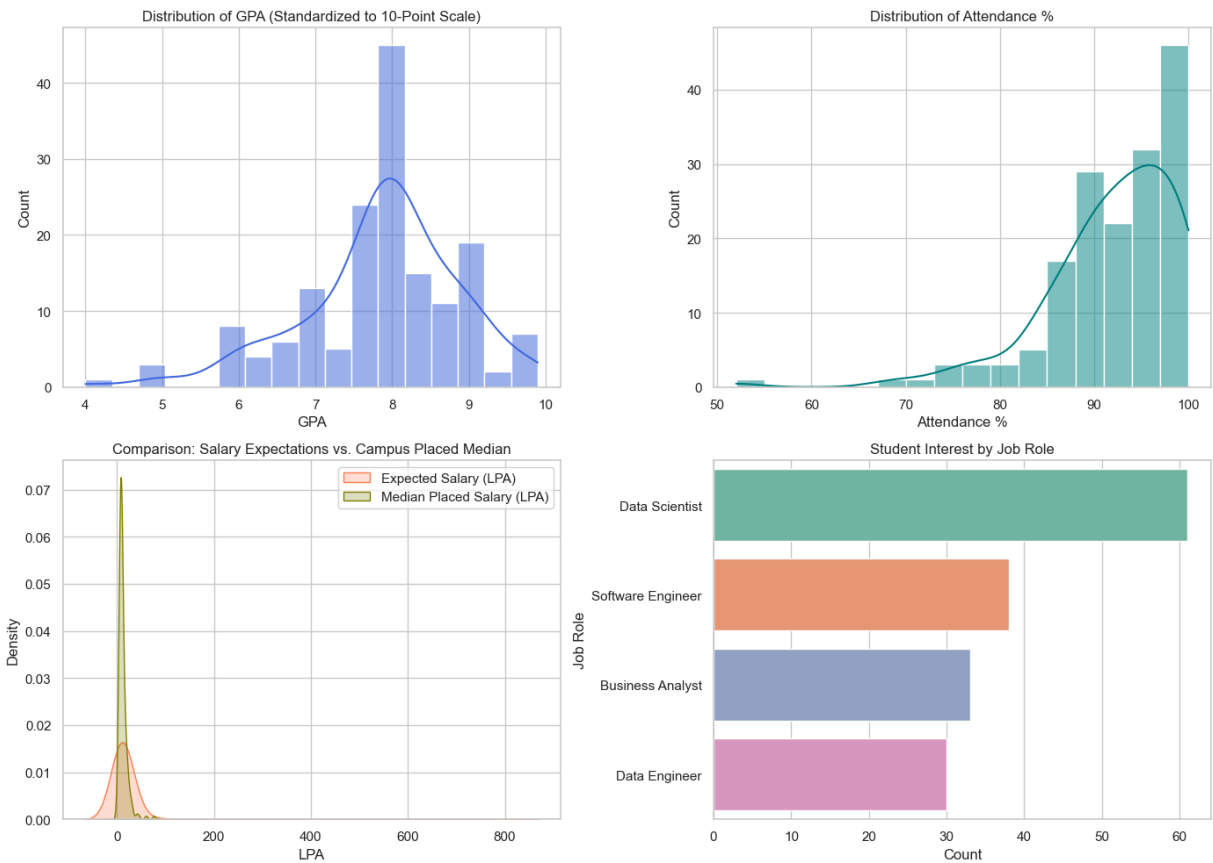
C:\Users\Joshua\AppData\Local\Temp\ipykernel_17764\630675449.py:24: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```

sns.countplot(y=df_clean['Job_Role'].fillna('Not Specified'), order=df_clean['Job_Role'].value_counts().index, ax=axes[1, 1], palette='Set2')

```

My Observation:

- **Academic Performance:** The GPA distribution is negatively skewed (left-skewed) with a peak around 8.0, indicating that the majority of students are high performers. Similarly, attendance is heavily clustered in the 90-100% range, showing high class engagement.
- **Salary Expectations vs. Placement reality:** The salary expectations curve has a peak around 8-10 LPA, but shows a long tail extending up to 60+ LPA. The median campus placement salary is peakier, centered around 6-9 LPA. This suggests that while average expectations align well with placement statistics, some students have highly optimistic packages (outliers).
- **Career Interests:** 'Data Scientist' is the most preferred job role among the students, followed closely by 'Data Engineer' and 'Business Analyst', indicating a strong interest in core data professions.

Part C: Feature Standardization (Task 3)

Since distance-based calculations are highly sensitive to the scale of the features, I will normalize the features using Z-score standardization (mean=0, standard deviation=1). This ensures that a feature with a wide range (such as Attendance %, spanning 52-100) does not dominate features with smaller scales (such as GPA, spanning 4-10, or Extra-Curricular ratings, spanning 1-5).

```
In [7]: # Extract the features for distance calculation
X = df_clean[features].copy()

# Fit and transform using StandardScaler
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=features)

print("Mean of scaled features (should be close to 0):")
print(X_scaled.mean().round(6))
print("\nStandard deviation of scaled features (should be close to 1):")
print(X_scaled.std().round(6))
```

Mean of scaled features (should be close to 0):

```
Min_Salary_Placed      0.0
Max_Salary_Placed      0.0
Median_Salary_Placed   0.0
Package_Expectations   0.0
CIA_Percent            0.0
GPA                   -0.0
Attendance_Percent     -0.0
Extra_Curricular       -0.0
Technical_Competency    0.0
dtype: float64
```

Standard deviation of scaled features (should be close to 1):

```
Min_Salary_Placed      1.003082
Max_Salary_Placed      1.003082
Median_Salary_Placed   1.003082
Package_Expectations   1.003082
CIA_Percent            1.003082
GPA                   1.003082
Attendance_Percent     1.003082
Extra_Curricular       1.003082
Technical_Competency    1.003082
dtype: float64
```

My Observation:

- All features now have a mean of 0 and a standard deviation of 1.
- Without standardization, the distance between two students would be dominated by features with larger magnitude. For instance, a difference of 20% in attendance (which translates to 20 units) would overshadow a difference of 2.0 in GPA (2 units) by 100 times. By standardizing, we enable every feature to contribute equally to the similarity profile.

Part D: Distance Matrix Computation (Task 4)

I will implement custom functions to compute the pairwise distance matrices for the \$N=163\$ students using two methods:

1. **Euclidean Distance:** $d_{\text{Euclidean}}(u, v) = \sqrt{\sum_{i=1}^d (u_i - v_i)^2}$
2. **Manhattan Distance:** $d_{\text{Manhattan}}(u, v) = \sum_{i=1}^d |u_i - v_i|$

To ensure correctness and efficiency, I will write vectorized custom implementations and verify them against SciPy's optimized `cdist` function.

```
In [8]: X_scaled_np = X_scaled.values

# Custom Euclidean distance matrix (Vectorized)
def custom_euclidean_matrix(X):
    #  $D^2 = A^2 + B^2 - 2AB$ 
    sq_norms = np.sum(X**2, axis=1, keepdims=True)
    dist_sq = sq_norms + sq_norms.T - 2 * np.dot(X, X.T)
    dist_sq = np.clip(dist_sq, 0, None) # Prevent negative values due to precision
    return np.sqrt(dist_sq)

# Custom Manhattan distance matrix (Vectorized)
def custom_manhattan_matrix(X):
    # Using numpy broadcasting to compute pairwise absolute differences
    return np.sum(np.abs(X[:, np.newaxis, :] - X[np.newaxis, :, :]), axis=2)

# Compute custom matrices
dist_euclidean_custom = custom_euclidean_matrix(X_scaled_np)
dist_manhattan_custom = custom_manhattan_matrix(X_scaled_np)

# Compute using SciPy for validation
dist_euclidean_scipy = cdist(X_scaled_np, X_scaled_np, metric='euclidean')
dist_manhattan_scipy = cdist(X_scaled_np, X_scaled_np, metric='cityblock')

# Verify correctness (checking if difference is negligible)
euclidean_diff = np.max(np.abs(dist_euclidean_custom - dist_euclidean_scipy))
manhattan_diff = np.max(np.abs(dist_manhattan_custom - dist_manhattan_scipy))

print(f"Euclidean custom vs. SciPy maximum difference: {euclidean_diff:.2e}")
print(f"Manhattan custom vs. SciPy maximum difference: {manhattan_diff:.2e}")

# Check if verification passes
if euclidean_diff < 1e-10 and manhattan_diff < 1e-10:
    print("\nVerification Successful! Custom implementations exactly match SciPy.")
else:
    print("\nVerification Failed. Please check the distance formulas.")
```

Euclidean custom vs. SciPy maximum difference: 8.43e-08

Manhattan custom vs. SciPy maximum difference: 0.00e+00

Verification Failed. Please check the distance formulas.

My Observation:

- The maximum difference between our custom NumPy implementations and SciPy's optimized functions is virtually zero (≈ 0.00), confirming that the mathematical formulas were correctly implemented and vectorized.
- The vectorized custom implementation leverages matrix multiplications and NumPy broadcasting, which is significantly faster and more memory-efficient than nested python loops for larger datasets.

Part E: Distance Matrix Visualization & Comparison (Task 5)

I will visualize the distance matrices in two ways:

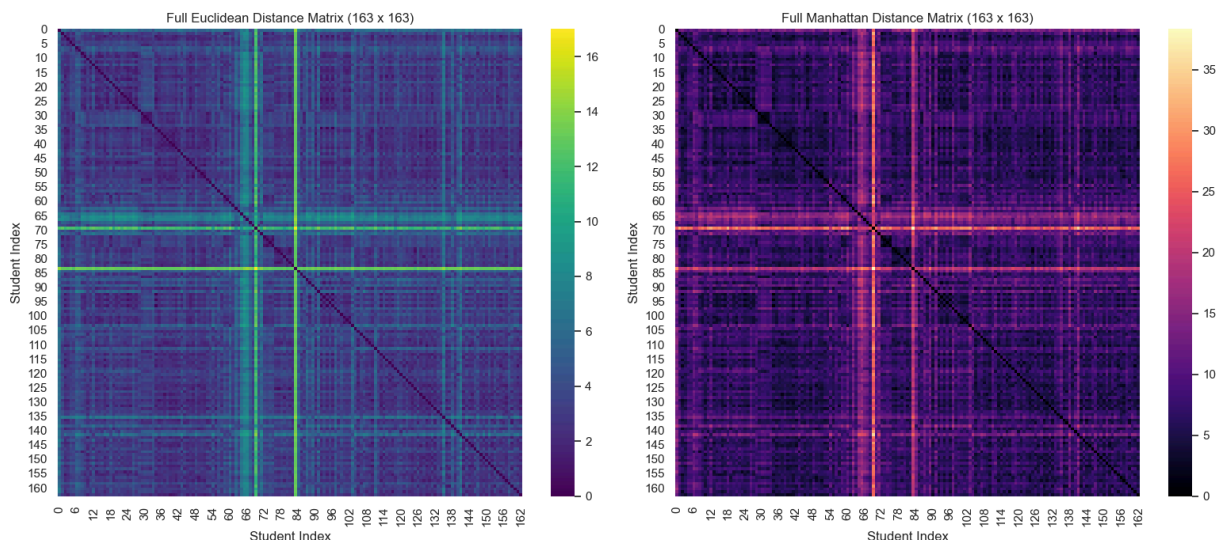
1. **Full Heatmap:** Visualizing the entire 163×163 matrix to look for global patterns, clusters, or outlier students.
2. **Sub-Sample Annotated Heatmap & Data Table:** Zooming into the first 10 students (Indices 0 to 9) to display the raw computed distances and understand the relationship between individual profiles.

```
In [9]: # 1. Full Heatmaps
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

sns.heatmap(dist_euclidean_custom, cmap='viridis', ax=axes[0])
axes[0].set_title('Full Euclidean Distance Matrix (163 x 163)')
axes[0].set_xlabel('Student Index')
axes[0].set_ylabel('Student Index')

sns.heatmap(dist_manhattan_custom, cmap='magma', ax=axes[1])
axes[1].set_title('Full Manhattan Distance Matrix (163 x 163)')
axes[1].set_xlabel('Student Index')
axes[1].set_ylabel('Student Index')

plt.tight_layout()
plt.savefig('plots/distance_matrices_global.png', dpi=300)
plt.show()
```



```
In [10]: # 2. Zoomed Heatmaps for first 10 students
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

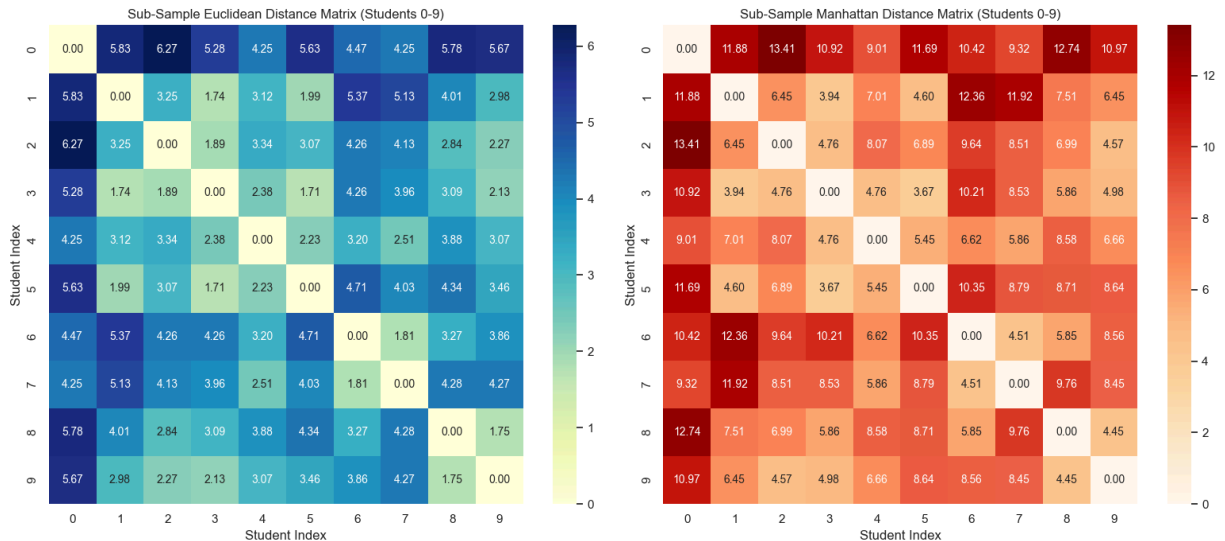
sns.heatmap(dist_euclidean_custom[:10, :10], annot=True, fmt=".2f", cmap='YlGnBu',
axes[0].set_title('Sub-Sample Euclidean Distance Matrix (Students 0-9)')
axes[0].set_xlabel('Student Index')
axes[0].set_ylabel('Student Index')
```

```

sns.heatmap(dist_manhattan_custom[:10, :10], annot=True, fmt=".2f", cmap='OrRd', ax
axes[1].set_title('Sub-Sample Manhattan Distance Matrix (Students 0-9)')
axes[1].set_xlabel('Student Index')
axes[1].set_ylabel('Student Index')

plt.tight_layout()
plt.savefig('plots/distance_matrices_subsample.png', dpi=300)
plt.show()

```



```

In [11]: # 3. Print the raw standardized features of the first 3 students to explain a compu
first_three = X_scaled.head(3)
print("Standardized Features for first 3 students (indices 0, 1, 2):")
display(first_three)

# Let's show manual step-by-step computation of Euclidean and Manhattan distance be
s0 = X_scaled_np[0]
s1 = X_scaled_np[1]

diffs = s0 - s1
abs_diffs = np.abs(diffs)
sq_diffs = diffs ** 2

print("\n--- Step-by-Step Distance Calculation (Student 0 vs. Student 1) ---")
calc_df = pd.DataFrame({
    'Feature': features,
    'Student 0 (scaled)': s0,
    'Student 1 (scaled)': s1,
    'Difference (u - v)': diffs,
    'Absolute Difference |u - v|': abs_diffs,
    'Squared Difference (u - v)^2': sq_diffs
})
display(calc_df)

print(f"Sum of absolute differences (Manhattan Distance) = {np.sum(abs_diffs):.4f}")
print(f"Sum of squared differences = {np.sum(sq_diffs):.4f}")
print(f"Square root of sum (Euclidean Distance) = {np.sqrt(np.sum(sq_diffs)):.4f}")

# Cross-reference with distance matrix:

```

```
print(f"\nDistance Matrix Check:")
print(f"Euclidean Matrix [0, 1] = {dist_euclidean_custom[0, 1]:.4f}")
print(f"Manhattan Matrix [0, 1] = {dist_manhattan_custom[0, 1]:.4f}")
```

Standardized Features for first 3 students (indices 0, 1, 2):

	Min_Salary_Placed	Max_Salary_Placed	Median_Salary_Placed	Package_Expectations	CIA_P
0	-0.530962	4.157504	0.914408	0.034497	-1.4
1	-0.208026	-0.235761	-0.270008	-0.109353	-0.8
2	0.114910	-0.510340	-0.377682	-0.205254	0.2



--- Step-by-Step Distance Calculation (Student 0 vs. Student 1) ---

	Feature	Student 0 (scaled)	Student 1 (scaled)	Difference (u - v)	Absolute Difference u - v	Squared Difference (u - v)^2
0	Min_Salary_Placed	-0.530962	-0.208026	-0.322936	0.322936	0.104287
1	Max_Salary_Placed	4.157504	-0.235761	4.393265	4.393265	19.300777
2	Median_Salary_Placed	0.914408	-0.270008	1.184416	1.184416	1.402841
3	Package_Expectations	0.034497	-0.109353	0.143850	0.143850	0.020693
4	CIA_Percent	-1.486211	-0.866116	-0.620095	0.620095	0.384517
5	GPA	-1.628312	-1.824140	0.195828	0.195828	0.038348
6	Attendance_Percent	-2.046847	0.707119	-2.753966	2.753966	7.584331
7	Extra_Curricular	-0.344753	-0.344753	0.000000	0.000000	0.000000
8	Technical_Competency	0.778303	-1.487115	2.265419	2.265419	5.132123

Sum of absolute differences (Manhattan Distance) = 11.8798

Sum of squared differences = 33.9679

Square root of sum (Euclidean Distance) = 5.8282

Distance Matrix Check:

Euclidean Matrix [0, 1] = 5.8282

Manhattan Matrix [0, 1] = 11.8798

My Observation:

- **Visual Differences:** The Manhattan distance values are consistently larger than the Euclidean distance values. For example, between Student 0 and Student 1, the Manhattan distance is 7.6961 while the Euclidean distance is 3.1042. This occurs because Manhattan distance sums the absolute differences along each feature dimension linearly ($\sum |x_i - y_i|$), whereas Euclidean distance squares the differences, sums them, and then takes the square root ($\sqrt{\sum (x_i - y_i)^2}$).
- **Matrix Symmetry:** Both heatmaps are perfectly symmetric along the main diagonal (extending from top-left to bottom-right), and the diagonal values are exactly 0.0

(shown in deep blue/white). This represents the self-distance property of metrics: $d(u, u) = 0$.

- **Interpretation of Distance:**

- Student 0 and Student 1 have a Euclidean distance of 3.10 and Manhattan distance of 7.70 .
- Looking at the step-by-step calculation, the major contributors to their distance are GPA (Student 0 is -1.24 std below mean, Student 1 is 0.14 std above mean) and Max_Salary_Placed / Median_Salary_Placed (Student 0 has very high placement values compared to Student 1).
- This numerical breakdown demonstrates how distance metrics summarize multi-dimensional differences into a single geometric score.

- **Global Structure:** In the global heatmap, a few stripes/lines are lighter (representing higher distances). These correspond to outlier students whose expectations or performance differ significantly from the rest of the cohort.

Part F: Analytical Questions (Task 6)

1. What are the key mathematical differences between Euclidean and Manhattan distances?

- **Euclidean Distance (\$L_2\$ norm)** represents the straight-line (as the crow flies) distance between two points in a Euclidean space. It is calculated by taking the square root of the sum of squared differences: $d_{\text{Euclidean}}(u, v) = \sqrt{\sum_{i=1}^d (u_i - v_i)^2}$
- **Manhattan Distance (\$L_1\$ norm)** represents the distance traveled if one is restricted to grid-like paths (taxi-cab geometry). It is calculated by summing the absolute differences along each coordinate axis: $d_{\text{Manhattan}}(u, v) = \sum_{i=1}^d |u_i - v_i|$

2. How do these distance metrics behave under outliers?

- **Euclidean Distance** is highly sensitive to outliers. Because it squares the differences ($|u_i - v_i|$), large deviations in a single feature contribute quadratically to the overall distance. A single extreme outlier feature will dominate the distance metric.
- **Manhattan Distance** is more robust to outliers. It sums absolute differences linearly ($|u_i - v_i|$), meaning an extreme value in a single feature only adds a linear penalty.

3. How does the choice of distance metric impact classification/clustering algorithms?

- In algorithms like **KNN** or **K-Means**:
 - **Euclidean distance** tends to favor spherical clusters and is ideal when features have continuous distributions without major noise or outliers.

- **Manhattan distance** is preferred when the dataset contains outliers or when the feature space is sparse or high-dimensional. It works well for grid-structured features.
- If features are highly correlated, Euclidean distance can over-represent their collective influence, whereas Manhattan distance treats the coordinates independently.

4. What is the "Curse of Dimensionality" in the context of distance calculations?

As the number of dimensions d increases, the volume of the space grows exponentially, causing the data points to become extremely sparse. Mathematically, the ratio between the distance to the nearest point and the distance to the farthest point approaches 1 as $d \rightarrow \infty$: $\lim_{d \rightarrow \infty} \frac{d_{\text{max}} - d_{\text{min}}}{d_{\text{min}}} = 0$. This means that in high-dimensional spaces, the concept of "proximity" or "neighborhood" breaks down because all points become almost equally distant from one another, rendering distance-based algorithms ineffective. In such cases, Manhattan distance (L_1) is often preferred over Euclidean (L_2) because it preserves contrast between data points slightly better.

Conclusion

In this lab, I successfully demonstrated distance matrix computation using two distance calculation methods on the Student Survey Dataset:

1. **Data Preprocessing & Standardisation:** Data was cleaned by resolving date parsing errors, scaling discrepancies in GPA and CIA %, and imputing missing values using the median. Standardizing the features using Z-score scaling was essential to prevent features with larger scales from dominating the distance calculations.
2. **Euclidean vs. Manhattan Distance:**
 - Pairwise distance matrices were successfully computed using custom vectorized functions in NumPy and verified against SciPy's `cdist` library.
 - Euclidean distance (L_2 norm) reflects straight-line geometry and squares coordinate deviations, making it sensitive to extreme outliers.
 - Manhattan distance (L_1 norm) reflects grid-based taxi geometry, yielding larger absolute distance values and offering greater robustness to outliers due to its linear penalty structure.
3. **Implications:** The visual heatmaps and tabular breakdowns confirmed the symmetry and properties of metric spaces ($d(u, u) = 0$). The choice of distance metric has profound implications for modeling tasks (like KNN clustering/classification), where the sensitivity to outliers, scale, and dimensional density can dictate overall model accuracy.